Semana 8: Matemáticas para ML II — probabilidad y estadística aplicada

Nivel: Intermedio | **Prerrequisitos:** Semana 7 (funciones como argumento, `pytest.approx`, idea de "error" a minimizar), Semana 6 (pruebas con `pytest`), Semana 4 (listas, diccionarios, list comprehensions), Semana 3 (funciones y bucles) | **Versión:** v1.0 | **Fecha:** 2026-10-12

Objetivos de aprendizaje

1. Calcular e interpretar la **media**, la **mediana** y la **desviación estándar** de una lista de datos, y explicar cuándo conviene cada medida.
2. Calcular la **probabilidad** de un evento como casos favorables sobre casos posibles, y **estimarla por simulación** con números aleatorios reproducibles.
3. Explicar la **ley de los grandes números**: por qué más repeticiones dan estimaciones más fiables.
4. Aplicar el **teorema de Bayes** para actualizar una creencia a partir de evidencia (ejemplo: filtro de spam).
5. Reconocer por qué los modelos de ML trabajan con probabilidades y por qué los datos se dividen en entrenamiento y prueba.

1. Por qué ML es, en el fondo, estadística con código

Un modelo de ML casi nunca "sabe" algo con certeza: **estima**. Un clasificador no dice "esto es un gato", dice "es un gato con 93 % de probabilidad". Los datos reales traen ruido, y la estadística es la herramienta para resumirlo y razonar sobre él.

- **Estadística descriptiva:** resumir muchos datos con pocos números.
- **Probabilidad:** medir cuán plausible es que algo ocurra.

Esta semana todo se programa en Python puro (con los módulos estándar `math` y `random`). Lo que viene de la Semana 7: ya sabes que entrenar es minimizar un error; ahora verás de dónde salen esos errores y cómo se mide la incertidumbre. Pandas y NumPy (Semana 9) harán esto mismo con una línea.

2. Resumir datos: media, mediana y desviación estándar

Analogía: el sueldo del barrio. En una calle viven cinco personas que ganan `[1, 2, 3, 4, 1000]` (en miles). Si llega un multimillonario, ¿cuál es el "sueldo típico"?

```
def media(valores: list[float]) -> float:
    """Promedio: suma de los valores dividida por cuántos hay."""
    if not valores:
        raise ValueError("La lista no puede estar vacía")   # fallar rápido, no dividir
entre cero
    return sum(valores) / len(valores)


def mediana(valores: list[float]) -> float:
    """Valor del medio una vez ordenados; resistente a valores extremos."""
    if not valores:
        raise ValueError("La lista no puede estar vacía")
    ordenados = sorted(valores)          # sorted() no altera la lista original (Semana
4)
    n = len(ordenados)
    medio = n // 2
    if n % 2 == 1:
        return ordenados[medio]
    return (ordenados[medio - 1] + ordenados[medio]) / 2


sueldos = [1, 2, 3, 4, 1000]
print(media(sueldos))     # 202.0 → arrastrada por el valor extremo
print(mediana(sueldos))   # 3     → el sueldo típico real
```

- La **media** usa todos los valores, pero un valor atípico (outlier) la distorsiona.
- La **mediana** es el valor central: ignora cuán extremos sean los extremos.

Ninguna dice cuán **dispersos** están los datos. Dos grupos pueden tener media 10 y ser muy distintos: `[10, 10, 10]` y `[2, 10, 18]`. Para eso se usa la **desviación estándar**: la distancia típica de los datos respecto a su media.

```
import math


def desviacion_estandar(valores: list[float]) -> float:
    """Distancia típica de los valores respecto a su media (poblacional)."""
    m = media(valores)
    # Se elevan al cuadrado las diferencias para que negativos y positivos no se
cancelen.
    varianza = sum((x - m) ** 2 for x in valores) / len(valores)
    return math.sqrt(varianza)   # la raíz devuelve el resultado a las unidades
originales


print(desviacion_estandar([10, 10, 10]))          # 0.0   → todos iguales
print(round(desviacion_estandar([2, 10, 18]), 3)) # 6.532 → muy dispersos
```

La **varianza** (el promedio de las diferencias al cuadrado) es el paso intermedio; la desviación estándar es su raíz. **Conexión con la Semana 7:** el "error cuadrático medio" que se minimizó con descenso del gradiente es exactamente una varianza de los errores.

3. Probabilidad: medir lo incierto

Analogía: el dado. La **probabilidad** de un evento es un número entre 0 (imposible) y 1 (seguro):

```
P(evento) = casos favorables / casos posibles      (si todos son igual de probables)
```

Un dado justo: `P(sacar 6) = 1/6 ≈ 0.1667`. Dos reglas que se usarán siempre:

- **Complemento:** `P(no ocurre) = 1 − P(ocurre)`.
- **Eventos independientes** (uno no afecta al otro): `P(A y B) = P(A) · P(B)`. Dos seises seguidos: `1/6 · 1/6 = 1/36`.

4. Simular en vez de calcular

No siempre hay una fórmula fácil. Un truco muy usado en ML y ciencia de datos: **repetir el experimento muchas veces con números aleatorios** y contar cuántas veces ocurre el evento (simulación de Monte Carlo).

```
import random


def probabilidad_simulada(evento, repeticiones: int = 100_000, semilla: int = 42) ->
float:
    """Estima P(evento) repitiendo un experimento aleatorio muchas veces."""
    azar = random.Random(semilla)        # generador propio con semilla: resultados
reproducibles
    exitos = sum(1 for _ in range(repeticiones) if evento(azar))
    return exitos / repeticiones


# ¿Probabilidad de sacar un 6? (teoría: 1/6 ≈ 0.1667)
print(probabilidad_simulada(lambda azar: azar.randint(1, 6) == 6))   # ≈ 0.1687

# ¿Y de que dos dados sumen 7? (teoría: 6/36 ≈ 0.1667)
print(probabilidad_simulada(lambda azar: azar.randint(1, 6) + azar.randint(1, 6) ==
7))   # ≈ 0.1646
```

Se pasa el experimento como función (Semana 7). La **semilla** (`semilla=42`) hace que el "azar" sea repetible: sin ella, tus pruebas darían resultados distintos cada vez. En ML, fijar semillas es práctica obligada para la **reproducibilidad** (Semana 6).

Ley de los grandes números

Mismo experimento (`P(6)`) con más repeticiones: 10 → `0.3`; 100 → `0.2`; 1.000 → `0.175`; 100.000 → `0.1687`. La estimación **se acerca al valor teórico (0.1667) a medida que aumentan las repeticiones**. Con pocos datos, el azar engaña. Por eso los modelos necesitan muchos ejemplos.

5. Probabilidad condicional y el teorema de Bayes

Analogía: el detective. Empiezas con una sospecha y, cada vez que aparece una pista, la ajustas. `P(A | B)` se lee "probabilidad de A **dado** B": cuán probable es A una vez que sabes que B ocurrió.

El **teorema de Bayes** convierte lo que sabes (`P(pista | clase)`) en lo que quieres saber (`P(clase | pista)`):

```
P(clase | pista) = P(pista | clase) · P(clase) / P(pista)
```

Ejemplo: filtro de spam. El 40 % del correo es spam (`P(spam) = 0.4`). La palabra "gratis" aparece en el 50 % del spam y en el 5 % del correo legítimo. Si llega un correo con "gratis", ¿cuán probable es que sea spam?

```
def bayes(p_clase: float, p_pista_dada_clase: float, p_pista_dada_otra: float) -> float:
    """P(clase | pista) por el teorema de Bayes, con dos clases posibles."""
    numerador = p_pista_dada_clase * p_clase
    # P(pista) = suma de ambos caminos: que venga de la clase o de la otra.
    total = numerador + p_pista_dada_otra * (1 - p_clase)
    return numerador / total


print(round(bayes(p_clase=0.4, p_pista_dada_clase=0.5, p_pista_dada_otra=0.05), 4))   # 0.8696
```

Pasó de 40 % a **87 %** con una sola pista. Un clasificador Naive Bayes hace esto con cientos de palabras a la vez; es uno de los modelos clásicos que verás en el Módulo 10.

6. Por qué esto importa para IA/ML

- **Los modelos producen probabilidades,** no certezas: la salida de un clasificador es una distribución sobre clases, y los modelos de lenguaje eligen la siguiente palabra según probabilidades (Módulo 12).
- **Dividir los datos:** si evalúas un modelo con los mismos datos con que lo entrenaste, es como poner el examen con las respuestas. Se separan en **entrenamiento** (para aprender) y **prueba** (datos nunca vistos, para medir). Un modelo que memoriza el entrenamiento pero falla en prueba sufre sobreajuste (overfitting).
- **Normalizar** (restar la media y dividir entre la desviación estándar) deja las características en una escala comparable, lo que ayuda al descenso del gradiente de la Semana 7.
- **Reproducibilidad:** fijar semillas aleatorias permite repetir y comparar experimentos.

7. Ejemplo práctico: ¿cuántas repeticiones necesito?

Problema: comprobar por simulación que la probabilidad de sacar al menos un 6 en 4 tiradas es mayor que 50 % (teoría: $1 - (5/6)^4 \approx 0.5177$, usando el complemento).

Pseudocódigo

```
repetir N veces:
    tirar un dado 4 veces
    si alguna salió 6: contar un éxito
mostrar éxitos / N
comparar con 1 − (5/6)^4
```

Código real (Python)

```
def al_menos_un_seis(azar) -> bool:
    return any(azar.randint(1, 6) == 6 for _ in range(4))   # any() se detiene al primer
True



simulada = probabilidad_simulada(al_menos_un_seis)
teorica = 1 - (5 / 6) ** 4
print(round(simulada, 4), round(teorica, 4))
```

Ambos valores quedan cerca de `0.52` (verificado por ejecución). Moraleja: cuando la fórmula exacta es difícil, la simulación es una verificación honesta.

Pruebas con pytest (Semana 6)

```
# test_estadistica.py
import pytest
from estadistica import media, mediana, desviacion_estandar, probabilidad_simulada,
bayes


def test_media():
    assert media([2, 4, 6]) == 4


def test_mediana_impar_y_par():
    assert mediana([5, 1, 3]) == 3
    assert mediana([4, 1, 3, 2]) == 2.5


def test_mediana_resiste_extremos():
    assert mediana([1, 2, 3, 4, 1000]) == 3
    assert media([1, 2, 3, 4, 1000]) == 202


def test_desviacion():
    assert desviacion_estandar([2, 4, 4, 4, 5, 5, 7, 9]) == pytest.approx(2.0)
    assert desviacion_estandar([5, 5, 5]) == 0


def test_lista_vacia():
    with pytest.raises(ValueError):
        media([])


def test_simulacion_dado():
    p = probabilidad_simulada(lambda azar: azar.randint(1, 6) == 6)
    assert p == pytest.approx(1 / 6, abs=0.01)


def test_bayes():
    assert bayes(0.4, 0.5, 0.05) == pytest.approx(0.2 / (0.2 + 0.03))
```

Las siete pruebas pasan al ejecutar `pytest` (verificado antes de publicar).

8. Notas de vigencia técnica (2026)

- **Módulo `statistics`:** la biblioteca estándar ya incluye `statistics.mean`, `median`, `pstdev` y `stdev`. Aquí se escriben a mano para entender qué calculan; en código real, usa el módulo estándar o, con datos grandes, NumPy/Pandas (Semana 9).
- **Desviación poblacional vs. muestral:** `pstdev` divide entre `n` (como esta clase); `stdev` divide entre `n - 1` y se usa cuando los datos son una muestra de una población mayor. Ambas son válidas; importa saber cuál usas.
- **Aleatoriedad:** `random.Random(semilla)` es adecuado para simulaciones y es reproducible, pero **no es criptográficamente seguro**: para contraseñas o tokens se usa `secrets`. En ciencia de datos con NumPy, la API vigente es `numpy.random.default_rng(semilla)`, no el antiguo `np.random.seed`.
- **Bayes hoy:** Naive Bayes sigue siendo una línea base rápida para texto; la inferencia bayesiana moderna usa librerías como PyMC. La idea de actualizar creencias con evidencia no ha cambiado.

9. Errores comunes de principiante

- Usar la media con datos que tienen valores atípicos extremos, sin mirar la mediana.
- Dividir entre `len(valores)` sin comprobar que la lista no esté vacía (`ZeroDivisionError`).
- Creer que tras varios fallos seguidos "ya toca" un acierto (falacia del apostador): los eventos independientes no tienen memoria.
- Olvidar fijar la semilla y no poder reproducir un resultado o una prueba.
- Sacar conclusiones de pocas repeticiones (la ley de los grandes números pide muchas).
- Confundir `P(A | B)` con `P(B | A)`: "casi todo el spam dice gratis" no significa "casi todo lo que dice gratis es spam".
- Evaluar un modelo con los mismos datos con que se entrenó.

10. Ejercicio propuesto

Escribe `normalizar(valores)` que devuelva la lista de `(x - media) / desviación_estandar` para cada valor (reutiliza `media` y `desviacion_estandar`). Comprueba con pytest que el resultado tiene media ≈ 0 y desviación estándar ≈ 1 (usa `pytest.approx`). Decide y prueba qué hacer cuando todos los valores son iguales (la desviación es 0). Versiona el resultado con Git en dos commits.

Pista: el caso de desviación 0 dividiría entre cero; elige entre lanzar un *`ValueError`* o devolver ceros, y justifícalo en un comentario.

11. Autoevaluación

1. ¿Cuándo es mejor la mediana que la media? — Cuando hay valores atípicos extremos, porque la mediana no se deja arrastrar por ellos.
2. Calcula la media y la mediana de `[1, 2, 3, 4, 10]`. — Media `4`, mediana `3`.
3. ¿Qué mide la desviación estándar? — Cuán dispersos están los datos respecto a su media; `0` significa que todos son iguales.
4. ¿Cuál es la probabilidad de sacar un número par en un dado justo? — `3/6 = 0.5`.
5. Si dos eventos son independientes con `P(A) = 0.5` y `P(B) = 0.2`, ¿cuánto vale `P(A y B)`? — `0.5 · 0.2 = 0.1`.
6. ¿Por qué se fija una semilla al simular? — Para que los resultados aleatorios sean reproducibles y las pruebas den lo mismo cada vez.
7. ¿Qué dice la ley de los grandes números? — Que la frecuencia observada de un evento se acerca a su probabilidad teórica al aumentar las repeticiones.
8. V/F: "`P(spam | gratis)` es lo mismo que `P(gratis | spam)`." — Falso; son distintas y el teorema de Bayes las relaciona.
9. Sin jerga: ¿por qué se separan datos de entrenamiento y de prueba? — Para medir al modelo con ejemplos que nunca vio; si no, solo comprobarías que memorizó.

12. Verificación de dependencias

Depende de la Semana 7 (pasar una función como argumento, que `probabilidad_simulada` usa para recibir el experimento; `pytest.approx` para comparar decimales; y la idea de error cuadrático medio, que aquí se reconoce como varianza), de la Semana 6 (pruebas con `pytest`, `pytest.raises` y versionado con Git en el ejercicio), de la Semana 4 (listas, list comprehensions, `sorted()` y la expresión generadora dentro de `sum`) y de la Semana 3 (funciones, `return` y bucles). La probabilidad y la estadística se introducen desde cero con aritmética básica; solo se importan los módulos estándar `math` y `random`, no se usa NumPy ni Pandas (Semana 9). Es prerrequisito directo de la Semana 9 (Python científico, que automatiza estos cálculos) y del Módulo 10 (Machine Learning clásico: Naive Bayes, evaluación con conjuntos de entrenamiento y prueba).